D5 - Automated Analysis Platform (SPBM): User and Audit Guide

Stockpile reconstruction and block model platform, version 2.0.0

> **Reproducibility standard.** A mine-planning engineer who did not develop the project should be able to install the software, reproduce every number in the report from the raw files, and add new data, using only this guide.

1. What the platform does

`spbm` is a Python package with a command-line interface. One command imports FMS dump records and survey change solids, checks them, reconstructs material movement (CPST algorithm, D3), builds the block model, quantifies uncertainty, validates the result, and writes an auditable record of the run.

Task 8 requirement	Where it happens	Output
1. Import new FMS records	`io.ingest` (managed raw store), `io.read_fms`	`data/raw_store/`, `audit/ingest_log.csv`, `audit/data_registry.csv`
2. Import new survey surfaces	`io.read_dxf_triangles`, `dataset.load_dataset`	cached grids in `data/cache/`
3. Data-quality checks	`rules.record_rules`, `qaqc.run_qaqc` (40-rule catalogue)	`qaqc/qa_rule_catalogue.csv`, `qaqc/*.csv`, `qaqc/fig_*.png`
4. New placement and modelling period	`io.survey_register` (cut-off D+1 07:00, interventions), `growth.temporal_alignment` (cut-off scan), `volume.calibrate` (volume factor)	`audit/survey_register.csv`, `growth/temporal_alignment.csv`, `growth/volume_factor_*.csv`
5. Material-movement algorithm	`movement.reconstruct`	`movement/load_destinations.csv`, `transport_plans.npz`
6. Update block model	`blockmodel.*`	`blockmodel/stockpile_block_model.csv`, `.vtk`
7. Tonnage and metal inventories	`blockmodel.summary_tables`, `validation.class_inventory`	`blockmodel/summary_*.csv`, `validation/inventory_by_class.csv`
8. Uncertainty	`uncertainty.monte_carlo`, `sensitivity_oat`, `scenarios`	`uncertainty/*.csv` (incl. `scenarios.csv`), block P10/P50/P90/CV
9. QA/QC and reconciliation outputs	`validation.*`	`validation/*.csv`, `growth/period_reconciliation.csv`
10. Visualisations and exports	`bmviz`, `growth`, `qaqc`	`*/fig_*.png`, `.vtk`, `.asc`, `report_tables.md`

Task 8 requirement	Where it happens	Output
11. Audit records	`audit.*`	`audit/run_manifest.json`, `run_log.csv`, `history/<run_id>/`, `change_report.md`

2. Installation

Requirements: Python ≥ 3.10 on Windows, macOS or Linux. No commercial software is needed. Results open in Excel (CSV), ParaView (VTK), any GIS (ESRI ASCII grids), and Vulcan/Datamine/Leapfrog (CSV block model with centroids and sizes).

```
git clone <repository>            # or unzip the submission
cd mine413
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m pytest -q               # 67 tests, about 3 min; must all pass
python -m pytest -q -m "not slow" # skip the end-to-end pipeline test (about 40 s)
```

Dependencies (all open source): numpy, scipy, pandas, matplotlib, ezdxf, pyyaml, openpyxl, pytest. Versions used are recorded in every `run_manifest.json`.

3. Input formats

Input	Format	Requirements
FMS	CSV with columns `Cycle ID, Truck PayLoad, Au, Ag, Cu, Dens, X (Dump Event), Y (Dump Event), Z (Dump Event), Dump_tstamp` (timestamp `M/D/YYYY H:MM`)	Any number of files with different names. Non-numeric values become missing and are caught by the QA rules. A repeated Cycle ID is flagged: DUP_IDENTICAL if the content is identical, DUP_CONFLICT otherwise. The first copy is modelled
Surveys	DXF with 3DFACE entities forming a closed change solid (Vulcan "STOCK_HGHS" export)	File name must contain the nominal date as `YYMMDD` (e.g. `Topo_260402.dxf`). The survey closes its interval at D+1 07:00 (`time.survey_cutoff_hours: 31`). Wrong dates or times are corrected by a manual intervention, never by renaming. The DXF header units are ignored (QA rule DXF_UNITS_HEADER); coordinates must be metres in the mine grid
Configuration	`config/project.yaml`	All parameters, each tagged with its assumption ID
Manual interventions	`config/manual_interventions.csv` (`id, type, target, value, reason, author, date, approved_by`)	See Section 4.3. A file with a missing reason, author or date, an unknown type or a repeated id is **rejected** and the run stops

Sources are listed in `inputs.sources` (default: the original `Data_Topo 2026.zip` and the folder `data/inbox/`). Every source file is copied once into the **managed raw store** `data/raw_store/`, named `<first 16 hex of SHA-256>__<name>`, and logged in `data/raw_store/ingest_log.csv`. The analysis reads only from the store. Ingesting is idempotent: a file already in the store is skipped. A new file with the **same name** as a stored file but **different content** is refused, unless a `supersede` intervention authorises it. The superseded version stays in the store, so the history of every input is kept.

4. Operating instructions

4.1 Reproduce the submitted analysis

```
        python -m spbm run                # full analysis, about 35 min (60 Monte Carlo
realisations)
        python -m spbm run --fast         # about 30 s: base reconstruction, block model,
QA/QC, balances only
        python -m spbm run --mc 200       # more realisations
```

Results are written to `outputs/`. The run is deterministic: the base case has no randomness, and the Monte Carlo uses the fixed seed in the configuration. Re-running reproduces identical numbers, which you can check by comparing `outputs/history/*/run_manifest.json`.

4.2 Routine update when new data arrive (Task 8)

1. Copy the new FMS export(s) and the new survey solid(s) into `data/inbox/`, or run `python -m spbm ingest <file or folder or zip>`.
2. If a survey file name is wrong, or a revised FMS export replaces an earlier file of the same name, add a row to `config/manual_interventions.csv` (Section 4.3).
3. Run

   ```
           python -m spbm update
   ```

   The platform lists the files it has not seen before (by SHA-256), re-runs the complete chain, and writes `outputs/audit/change_report.md` comparing tonnes, metal and classes with the previous model version.
4. Review, in this order:
   - `qaqc/qa_rule_catalogue.csv` (counts per rule, at record, survey-file and dataset level);
   - `growth/temporal_alignment.csv` (the run notes if the cut-off scan moves away from 31 h);
   - `growth/volume_factor_calibration.csv` (a new interval with a ratio far from f_v is an FMS gap or a survey problem);
   - `growth/period_reconciliation.csv` (large unattributed or retained volumes mean the FMS and the survey disagree);
   - `validation/geometry_check.csv`;

    - then the block model.

5. Approve the model version by recording the `run_id` in the decision log. Loads after the latest survey are flagged *Provisional*. They will be reconciled automatically once the next survey is imported.

Why a full re-run instead of an incremental patch? A full run takes minutes, gives identical results for identical inputs, and lets late corrections (e.g. a re-dated survey or a corrected FMS record) propagate correctly through all later periods. Incremental state files would be faster but harder to audit.

4.3 Manual interventions

Raw files are never edited. A correction is a row in `config/manual_interventions.csv`:

Column	Content
`id`	Unique identifier, e.g. `MI-003`
`type`	`survey_date` (value = corrected nominal date YYYY-MM-DD), `survey_cutoff_hours` (value = hours after 00:00 of the nominal date), `fms_exclude` (target = Cycle ID; record stays in the register but is not modelled), `fms_location_unknown` (target = Cycle ID; dump X/Y treated as unknown), `supersede` (target = file name, value = first 12 characters of the new SHA-256)
`target`	Survey file name, Cycle ID or file name
`value`	As above
`reason`	Required. The evidence for the change
`author`, `date`	Required
`approved_by`	Reviewer (recommended)

The two rows supplied (MI-001, MI-002) re-date `Topo_250328.dxf` and `Topo_250329.dxf` to 2026 (QA rule SURVEY_NAME_YEAR). Applied interventions are written to `audit/manual_interventions_applied.csv` and into the run manifest. Their ids appear in `audit/survey_register.csv` next to the file they changed.

4.4 Example execution: demonstrating an update

The repository includes a demonstration of the update workflow. A model is built with the data available at the 03-22 survey cut-off (2026-03-23 07:00), then updated with the remaining data (`examples/demo_update.sh`):

```
python -m spbm run --fast --until "2026-03-23 07:00" --out outputs_demo
python -m spbm run --fast --out outputs_demo
cat outputs_demo/audit/change_report.md
```

The automated test `tests/test_platform.py::test_incremental_update_reconciles_pending_loads` checks the key property of an update. Loads that were provisional (after the last survey) in the earlier model are survey-reconciled in the later one.

4.5 Other commands

```
        python -m spbm ingest PATH ...   # add files, folders or zips to the managed raw store
        python -m spbm qaqc      # data checks only, prints the flag summary
        python -m spbm status    # registered input files and run log
        python tools/make_diagrams.py   # redraw workflow/algorithm diagrams
        python tools/build_docs.py      # rebuild DOCX/PDF documents and the presentation
        python tools/fill_report.py     # refresh D4/D6 numbers from the latest run (templates
in tools/templates)
        python tools/make_submission.py   # Canvas zip in dist/ (team name from
config/team.yaml)
```

Team names: edit `config/team.yaml`. Every `[Member n]` placeholder in the built documents and the presentation is replaced at build time.

Document tooling (optional; not needed for the analysis): pandoc, Node.js with `npm install` in `tools/` (pptxgenjs), and Playwright's Chromium for PDF rendering.

5. Output reference

File	Content
`blockmodel/stockpile_block_model.csv`	One row per block containing material. See D3 Section 5 for fields
`blockmodel/stockpile_block_model.vtk`	Same, as a VTK structured grid (ParaView: *Threshold* on `tonnes > 0`)
`blockmodel/block_model_definition.json`	Origin, block size, units, CRS note
`blockmodel/summary_*.csv`	Inventory by material source (**exact** tonnage split by class), by dominant information class (whole blocks labelled by their dominant class, for maps), confidence, 4 m bench; grade-tonnage
`movement/load_destinations.csv`	For each load: dump X/Y, volume placed, retained, destination centroid, mean push distance, spread
`movement/transport_plans.npz`	Transport matrix of every period (full traceability of the allocation)
`movement/*.asc`	Floor plane, pre-record surface, final model surface, latest surveyed surface (ESRI ASCII grids)
`qaqc/`	QA rule catalogue, records with one column per rule, record-rule summary, daily coverage by production day, grade signatures, survey QA (incl. DXF header units and dates), figures
`growth/`	Period reconciliation, survey cut-off scan and dump time-of-day histogram, volume factor calibration and hold-out, growth figures
`validation/`	Mass/metal balance, inventory by class, volume reconciliation, geometry, forward-model temporal validation, twin experiment, robustness

File	Content
`uncertainty/`	Monte Carlo realisations and global summary, tornado table, volume-gap scenarios (`scenarios.csv`, `fig_scenarios.png`), figures
`report_tables.md`, `summary.json`	Key numbers for the report
`audit/`	Data registry, ingest log, survey register, manual interventions applied, run manifest, configuration used, run log, change report
`history/<run_id>/`	Archived manifest and key tables for every model version

6. Audit and reproducibility controls

- **Input identity**: every input file is hashed (SHA-256), copied once into the managed raw store and logged with the date it was ingested and where it came from. A changed file with an existing name is refused unless a `supersede` intervention authorises it. The manifest lists the hashes used by the run.
- **Configuration identity**: the configuration is hashed and copied (`config_used.json`). Every parameter is linked to an assumption ID.
- **Code identity**: the Git commit (`+modified` if the source differs from it), a SHA-256 of the platform source and configuration (`spbm_source_sha256`), and all library versions are recorded. In an unzipped submission package without `.git`, the commit is read from `BUILD_INFO.json`, written by `tools/make_submission.py`.
- **Manual interventions**: only through `manual_interventions.csv`. Each row has an id, a type, a reason, an author and a date, and the file is rejected otherwise. Interventions are copied into the manifest. Raw files are never edited.
- **No silent deletion**: FMS records are flagged, never removed. Each QA rule states its treatment. Only error-severity rules (missing or invalid fields, repeated copies, dumps before the record) exclude a record from modelling, and the record stays in `fms_records_with_flags.csv`.
- **Model versions**: `history/` keeps each version's manifest and key tables; `change_report.md` documents differences.
- **Verification**: `python -m pytest` runs 67 tests in three files.
  - `tests/test_units.py` covers synthetic inputs: rasterisation, the transport solver, column operations, avalanche, DXF reading, every QA record rule, ingest idempotence and conflict refusal, intervention validation, the survey cut-off, robust volume-factor estimation and the density identity.
  - `tests/test_real_data.py` pins the facts the report relies on: record counts, rule counts, DXF header units, the cut-off scan optimum, the shift-change lull, the calibrated f_v and its hold-out error, mass and metal conservation, and the record-period inventory.
  - `tests/test_platform.py` covers the command line, interventions in the engine, the manifest and change report, the incremental update, and an end-to-end run in a temporary folder.

7. Usability for different levels of expertise

- **Planners / managers**: read `report_tables.md` and the PNG figures. Use the block model CSV in Excel or the mine-planning package. Filter by `confidence` and `info_class`.
- **Engineers**: change parameters in `config/project.yaml` (comments explain each one and link to D2) and re-run.
- **Developers**: modules are small and documented; the tests define the expected behaviour.

8. Known limitations of the platform

- Survey inputs must be closed change solids (the format supplied). Open surfaces are detected (QA reports `open_cells`) but are not yet converted automatically. Recommended extension: accept full-pile surfaces and compute the change solids internally.
- Single-stockpile configuration. Several stockpiles would need one configuration each.
- There is no graphical user interface (not required). Outputs are files and figures.